EduNote

Kiet Chu, Beatrice Calvelo, Bao Dao, Anh Tran

Objective

Our project is an AI-powered tutoring and smart notebook application designed to help students learn more effectively. It integrates note-taking with real-time AI assistance, providing explanations, summaries, and problem-solving guidance directly within the notebook environment.

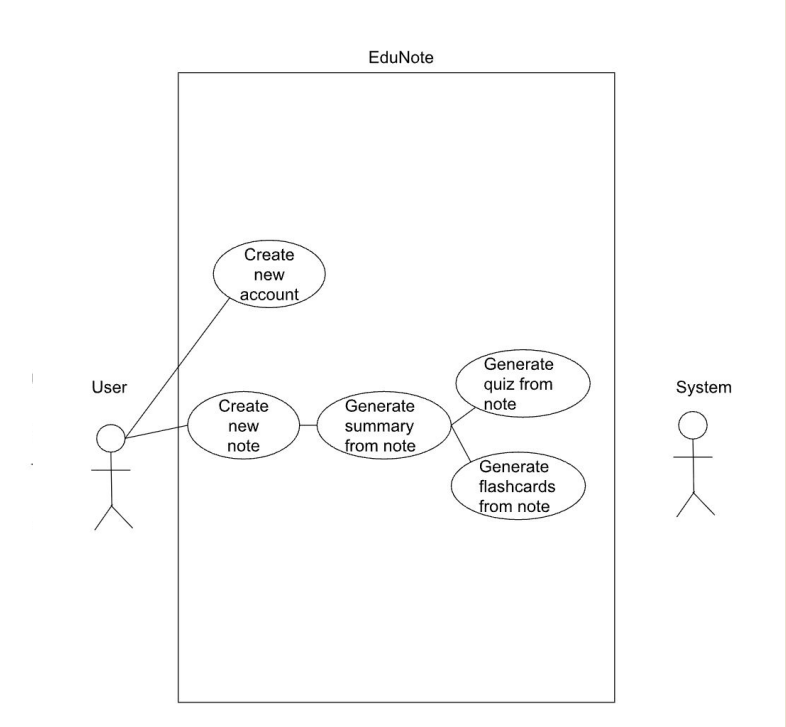
Motivation

Students often juggle between lecture notes, online resources, and Q&A platforms while studying. This fragmented workflow reduces focus and learning efficiency. Our system addresses this gap by combining note-taking and tutoring into a single, intelligent interface. It benefits students by providing instant clarifications, interactive feedback, and personalized study support.

High-Level Features

- User Registration and Login
- Note Editor
- AI Summary
- Quiz Generator
- Flashcard Generator
- AI Chat Bot

Use Case Diagram



Creating A New Account Case

- Actors: Student
- Precondition: Student opens the front page
- Main Flow:
  1. Student selects ‘Sign up’.
  2. System opens up the ‘Sign up’ page.
  3. Student enters their credentials and registers.
  4. System saves their credentials and creates a new account for the user.
- Alternative Flows: If the account credentials already exist, the system will not create a new account and send a warning that it is unable to create a new account.
- Postconditions: User credentials are stored and a new account is created.

Creating A New Note Case

- Actors: Student
- Precondition: Student is logged in
- Main Flow:
  1. Student selects the New Note button or clicks on an existing note.
  2. For creating a new note, student can upload a new pdf.
  3. System opens a split view: pdf viewer on left, notes pane on right.
  4. Student creates a note and system links the note with the uploaded pdf.
  5. System saves both the pdf file and the notes.
- Alternative Flows: If the pdf size is bigger than the size limit, the file will not be saved and the system will notify the user.
- Postconditions: File is stored along with the metadata.

Generate Summary from Note Case

- Actors: Student
- Precondition: Student is logged in and opens a note
- Main Flow:
  1. Student selects a note.
  2. System opens a split view: pdf viewer on left, note pane on right.
  3. Student selects ‘Generate Summary’ from the action bar.
  4. System sends an API call to OpenAI and receives a generated summary.
  5. System saves and displays the generated summary.
- Alternative Flows: If the OpenAI API doesn’t respond, the system will notify the user.
- Postconditions: Summary is stored along with the selected note.

Generate Quizzes from Note Case

- Actors: Student
- Precondition: Student is logged in and opens a note
- Main Flow:
  1. Student selects a note.
  2. System opens a split view: pdf viewer on left, note pane on right.
  3. Student selects ‘Generate Quizzes’ from the action bar.
  4. System sends an API call to OpenAI and receives a generated quizzes.
  5. System saves and displays the generated quiz.
- Alternative Flows: If the OpenAI API doesn’t respond, the system will notify the user.
- Postconditions: Quiz is stored along with the selected note.

Generate Flashcards from Note Case

- Actors: Student
- Precondition: Student is logged in and opens a note
- Main Flow:
  1. Student selects a note.
  2. System opens a split view: pdf viewer on left, note pane on right.
  3. Student selects ‘Generate Flashcards’ from the action bar.
  4. System sends an API call to OpenAI and receives a generated flashcards.
  5. System saves and displays the generated flashcards.
- Alternative Flows: If the OpenAI API doesn’t respond, the system will notify the user.
- Postconditions: Flashcards are stored along with the selected note.

Chat With AI Case

- Actors: Student
- Precondition: Student is logged in and opens a note
- Main Flow:
  1. Student selects a note.
  2. System opens a split view: pdf viewer on left, note pane on right.
  3. Student selects ‘Chat’ from the action bar.
  4. Student enters their question in the textbox and presses ‘Send’.
  4. System sends an API call to OpenAI and receives an answer.
  5. System saves and displays the generated answer.
- Alternative Flows: If the OpenAI API doesn’t respond, the system will call the offline LLM model.
- Postconditions: A chat history is saved and the ChatID is associated with the note.

Feasibility & Scope

- Frontend: React for notebook interface.
- Backend: FastAPI/ Flask for integration with OpenAI API.
- AI Model: OpenAI API for summarizing documents and generating quizzes.
- Database: Postgres for document and user metadata.
- Cloud: Google Cloud for storing uploaded documents.

Preliminary Timeline

- Week 1 - 2: Finalize system architecture and setup repo + cloud environment.
- Week 3 - 4: Implement user login + note editor.
- Week 5: Implement PDF parser.
- Week 6 - 7: Implement AI summary.
- Week 8 - 9: Implement flashcard and quiz generator.
- Week 10 - 11: Polish user interface.
- Week 12: Fix bugs.
- Week 13: Final presentation and documentation.

Team Contributions

- Frontend: Kiet Chu
- Backend: Beatrice Calvelo, Bao Dao
- OpenAI API + Database: Anh Tran



Ask us anything!